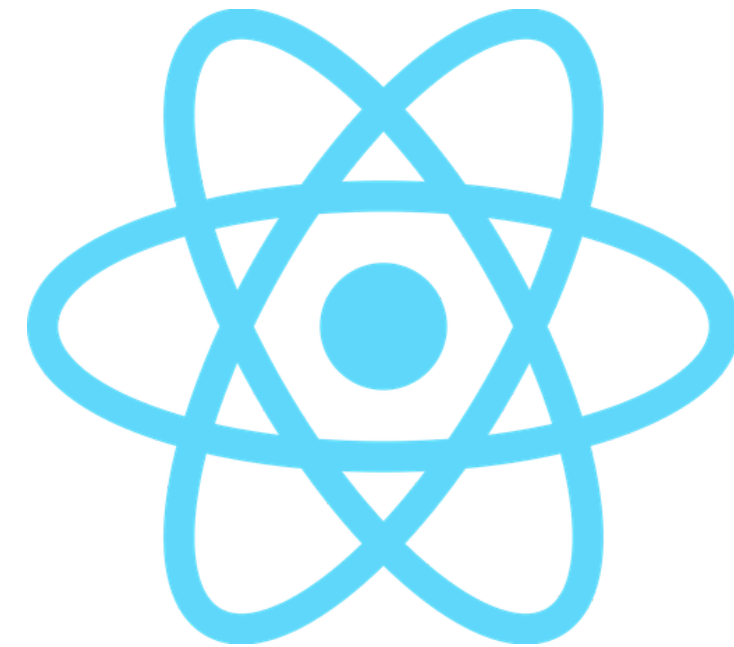


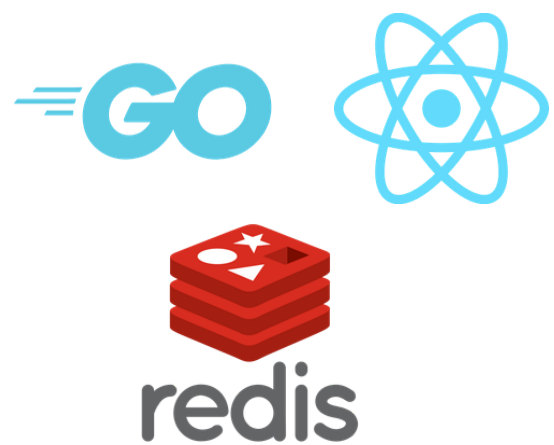
Présentation

Golang - ReactJS

Axel Pointud — M2 ESGI - 16 septembre 2026

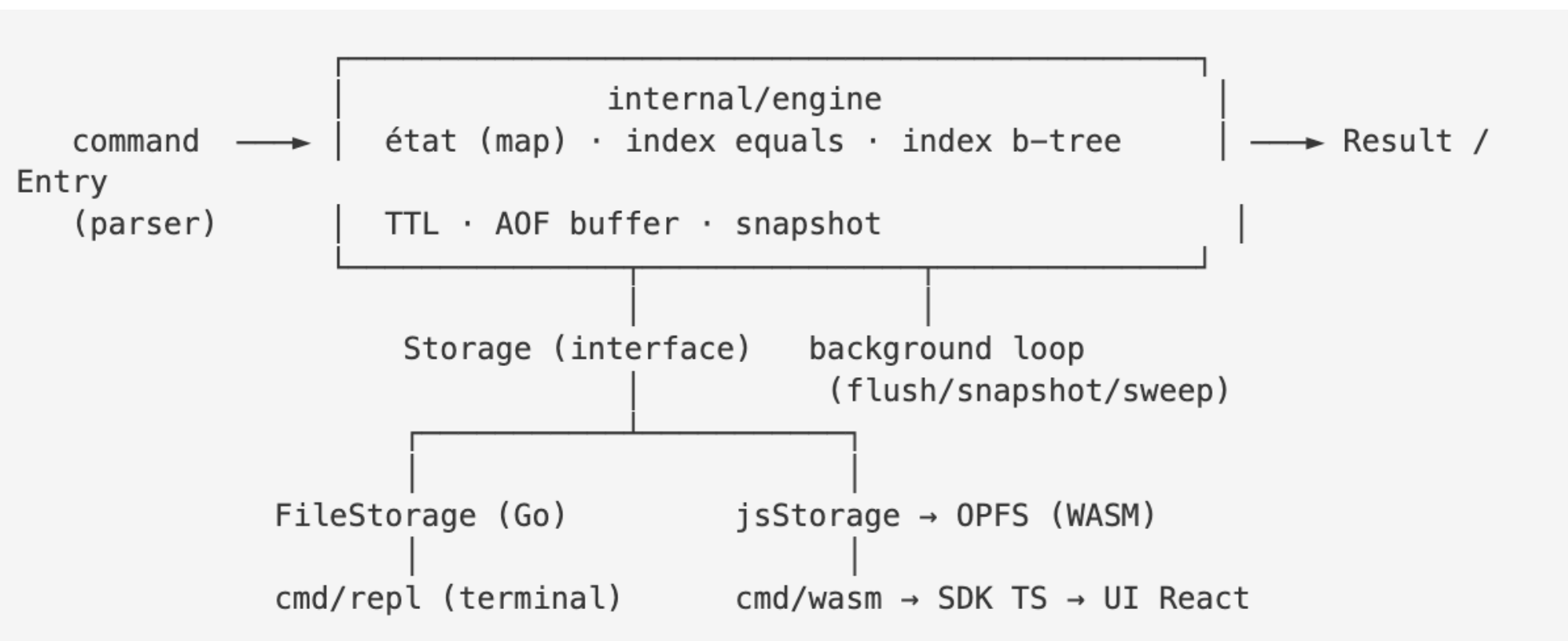
Code source : <https://github.com/axel-po/wasm-golang-redis>



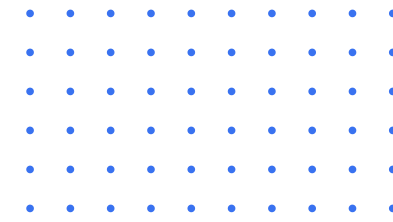


Projet Redis LIKE

Un magasin clé/valeur en mémoire, écrit en Go, construit par étapes. On part d'un moteur minimal piloté par un REPL (Read-Eval-Print Loop), puis on ajoute la persistance sur disque, l'exécution par lots, les requêtes filtrées avec index, l'expiration des clés (TTL), et enfin une compilation en WebAssembly exposée via un SDK TypeScript et une interface web React.



Sommaire



Partie 1 — Le moteur Redis en Go

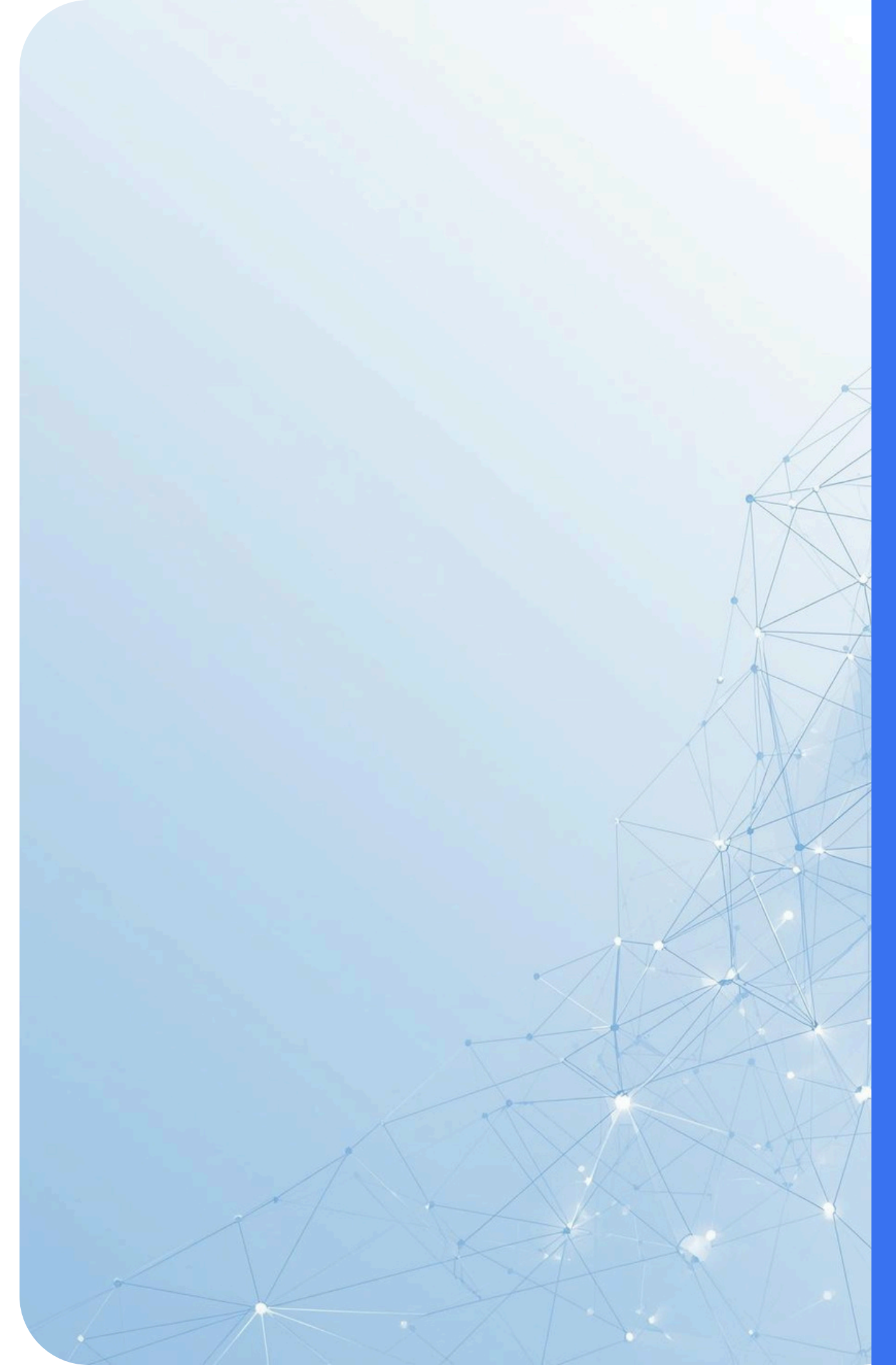
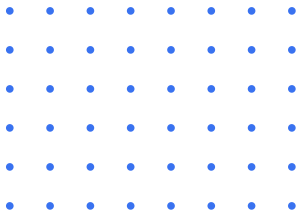
- Moteur + parser + REPL (commandes typées)
- Persistance : AOF, snapshots & restore
- Recherche par valeur : GET WHERE, index & B-Tree
- Expiration des clés (TTL)

Partie 2 — Du Go au navigateur (React)

- Compilation WebAssembly & SDK TypeScript
- Le virtual scrolling : afficher 100 000 clés sans ramer

Partie 3 — Démo live

- Le même moteur, du terminal au navigateur



Moteur + parser + REPL

Les commandes typées (Command, Set, Get, Delete)

Chaque commande Redis est un type Go → le compilateur vérifie tout

On représente chaque commande Redis par un type Go distinct : Set, Get, Delete, GetWhere.

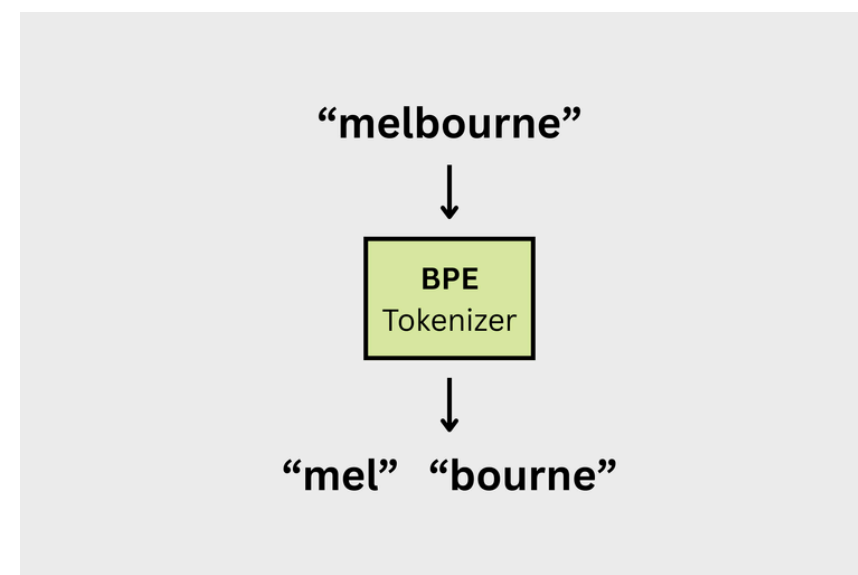
```
1  type Command interface {
2      isCommand()
3      String() string
4  }
5
6  type Set struct {
7      Key   string
8      Value string
9      TTL   time.Duration
10 }
11
12 type Get struct {
13     Key string
14 }
15
16 type Delete struct {
17     Key string
18 }
19
20 type Operator string
21
22 const (
23     OpEquals Operator = "equals"
24     OpContains Operator = "contains"
25     OpGT      Operator = ">"
26     OpGTE     Operator = ">="
27     OpLT      Operator = "<"
28     OpLTE     Operator = "<="
29 )
30
31 type GetWhere struct {
32     Op      Operator
33     Value string
34 }
35
36 func (Set) isCommand() {}
37 func (Get) isCommand() {}
38 func (Delete) isCommand() {}
39 func (GetWhere) isCommand() {}
```



Moteur + parser + REPL

Un tokenizer qui gère les guillemets pour les valeurs à espaces

On découpe la ligne tapée en mots, en gérant les guillemets pour autoriser les espaces dans une valeur.



Le parsing commence par un tokenizer qui découpe la ligne en mots, mais qui gère les guillemets : SET msg "hello world" doit donner une seule valeur, pas deux tokens.

```
1 package command
2
3 import "strings"
4
5 func tokenize(input string) ([]string, error) {
6     var (
7         tokens []string
8         current strings.Builder
9         inQuote bool
10        started bool
11    )
12
13    flush := func() {
14        if started {
15            tokens = append(tokens, current.String())
16            current.Reset()
17            started = false
18        }
19    }
20
21    for _, r := range input {
22        switch {
23            case r == '"':
24                inQuote = !inQuote
25                started = true
26            case (r == ' ' || r == '\t') && !inQuote:
27                flush()
28            default:
29                current.WriteRune(r)
30                started = true
31        }
32    }
33
34    if inQuote {
35        return nil, ErrUnclosedQuote
36    }
37    flush()
38
39    return tokens, nil
40 }
41
```

Moteur + parser + REPL

Fonction centrale Apply qui exécute une commande

Le pattern matching relie chaque commande à son opération

```
1 func (e *Engine) Apply(cmd command.Command) (Result, error) {
2     switch c := cmd.(type) {
3     case command.Set:
4         if c.TTL > 0 {
5             e.SetEX(c.Key, c.Value, c.TTL)
6         } else {
7             e.Set(c.Key, c.Value)
8         }
9         return Result{}, nil
10
11    case command.Get:
12        value, err := e.Get(c.Key)
13        if err != nil {
14            return Result{}, err
15        }
16        return Result{Value: value, Found: true}, nil
17
18    case command.Delete:
19        return Result{Found: e.Delete(c.Key)}, nil
20
21    default:
22        return Result{}, fmt.Errorf("apply %v: %w", cmd, ErrUnsupportedCommand)
23    }
24 }
```

Une fois la commande parsée, Apply fait le pattern matching : un switch sur le type concret aiguille vers la bonne méthode du moteur. C'est le seul point d'exécution : batch et WASM passent tous par là.

Une fois la commande parsée, Apply fait le pattern matching : un switch sur le type concret aiguille vers la bonne méthode du moteur. C'est le seul point d'exécution : batch et WASM passent tous par là.

 execute.go X

internal > engine >  execute.go >  (*Engine).Apply

Moteur + parser + REPL

La boucle interactive (REPL)

La boucle interactive du terminal : elle lit, exécute, et affiche (nil) ou OK exactement comme le vrai Redis



Le REPL, c'est la boucle read-eval-print : je lis, je parse, j'applique, j'affiche (nil), OK ou la valeur — exactement comme le vrai redis-cli. Le cas GetWhere est traité à part car il renvoie plusieurs entrées.

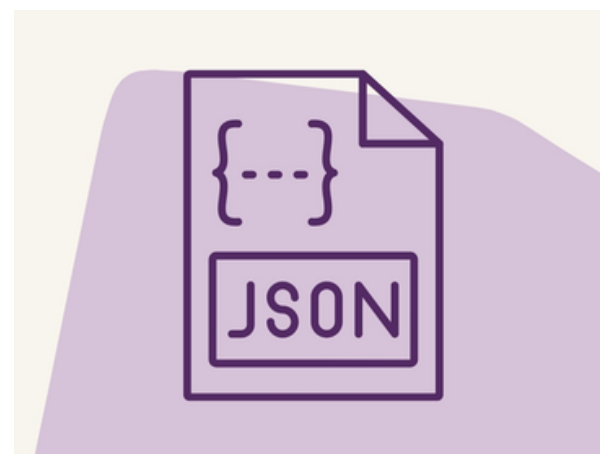
```
1  for {
2      fmt.Print("> ")
3      if !scanner.Scan() {
4          if err := scanner.Err(); err != nil {
5              fmt.Fprintln(os.Stderr, "erreur de lecture:", err)
6          }
7          return
8      }
9
10     cmd, err := command.Parse(scanner.Text())
11     if err != nil {
12         fmt.Println("ERR", err)
13         continue
14     }
15
16     if q, ok := cmd.(command.GetWhere); ok {
17         entries := db.Filter(q)
18         if len(entries) == 0 {
19             fmt.Println("(vide)")
20             continue
21         }
22         for _, entry := range entries {
23             fmt.Printf("%s = %q\n", entry.Key, entry.Value)
24         }
25         continue
26     }
27
28     result, err := db.Apply(cmd)
29     switch {
30     case errors.Is(err, engine.ErrKeyNotFound):
31         fmt.Println("(nil)")
32     case err != nil:
33         fmt.Println("ERR", err)
34     case result.Found:
35         fmt.Printf("%q\n", result.Value)
36     default:
37         fmt.Println("OK")
38     }
```

Persistance (AOF + Snapshots)

Le journal des écritures (AOF)

Chaque écriture est enregistrée sous forme d'une petite ligne JSON : c'est le journal, l'Append-Only File.

```
1 package engine
2
3 import "encoding/json"
4
5 type opKind string
6
7 const (
8     opSet    opKind = "set"
9     opDelete opKind = "delete"
10 )
11
12 type Operation struct {
13     Kind    opKind `json:"kind"`
14     Key     string `json:"key"`
15     Value   string `json:"value,omitempty"`
16     ExpiresAt int64 `json:"expires_at,omitempty"`
17 }
18
19 func encodeOp(op Operation) ([]byte, error) {
20     return json.Marshal(op)
21 }
22
23 func decodeOp(line []byte) (Operation, error) {
24     var op Operation
25     err := json.Unmarshal(line, &op)
26     return op, err
27 }
28
```



Passons à la persistance. La RAM est volatile : si l'onglet ferme, tout est perdu. Premier mécanisme : l'Append-Only File. Chaque écriture est sérialisée en une ligne JSON — set, la clé, la valeur, l'expiration — et ajoutée au journal. C'est l'historique de tout ce qui s'est passé.

Persistance (AOF + Snapshots)

L'interface qui change tout

Le moteur ne sait pas où il écrit : il parle à une interface. C'est ce qui me permettra plus tard d'écrire dans le navigateur.

```
1 package engine
2
3 type Storage interface {
4     AppendLog(lines [][]byte) error
5     ReadLog() ([][]byte, error)
6     ClearLog() error
7     WriteSnapshot(data []byte) error
8     ReadSnapshot() ([]byte, error)
9 }
10
```

Point d'architecture clé : le moteur ne sait pas où il écrit. Il parle à une interface Storage — append au log, lire, vider, écrire/lire le snapshot. C'est ce découplage qui me permettra, sans toucher au moteur, de brancher les fichiers en local et l'OPFS dans le navigateur.

 storage.go ×

internal > engine >  storage.go > ...

Persistance (AOF + Snapshots)

Écrire par lots, pas à chaque coup

On n'écrit pas sur le disque à chaque commande : on accumule dans un **buffer**, puis on **flushe** d'un coup. C'est plus rapide.

```
1 package engine
2
3 func (e *Engine) Flush() error {
4     e.mu.Lock()
5     defer e.mu.Unlock()
6
7     if e.storage == nil || len(e.buffer) == 0 {
8         return nil
9     }
10
11     lines := make([][]byte, 0, len(e.buffer))
12     for _, op := range e.buffer {
13         line, err := encodeOp(op)
14         if err != nil {
15             return err
16         }
17         lines = append(lines, line)
18     }
19
20     if err := e.storage.AppendLog(lines); err != nil {
21         return err
22     }
23
24     e.buffer = e.buffer[:0]
25     return nil
26 }
27
```

On n'écrit pas sur disque à chaque SET — trop lent. On accumule dans un buffer en RAM, et une fois par seconde on flushe tout d'un coup. Le mutex garantit un seul flush à la fois. Au pire, on perd 1 seconde de données en cas de crash : c'est le compromis vitesse/durabilité.

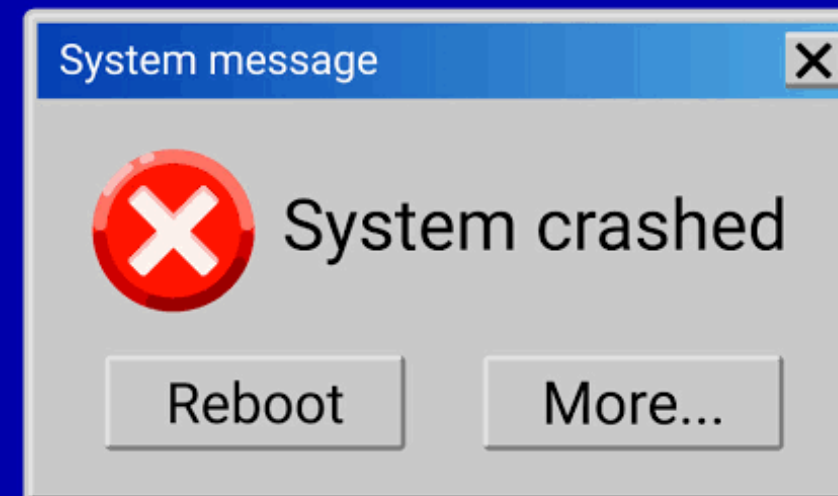
Persistence (AOF + Snapshots)

Renaître après un crash

Au démarrage on recharge le dernier snapshot, puis on rejoue le journal par-dessus pour retrouver l'état exact.

```
1 func (e *Engine) Restore() error {
2     e.mu.Lock()
3     defer e.mu.Unlock()
4
5     if e.storage == nil {
6         return nil
7     }
8
9     snap, err := e.storage.ReadSnapshot()
10    if err != nil {
11        return err
12    }
13    if snap != nil {
14        var dump map[string]persistedRecord
15        if err := json.Unmarshal(snap, &dump); err != nil {
16            return err
17        }
18        for key, pr := range dump {
19            e.state[key] = record{value: pr.Value, expiresAt: fromUnixNano(pr.ExpiresAt)}
20        }
21    }
22
23    lines, err := e.storage.ReadLog()
24    if err != nil {
25        return err
26    }
27    for _, line := range lines {
28        op, err := decodeOp(line)
29        if err != nil {
30            return err
31        }
32        e.applyOp(op)
33    }
34    e.rebuildIndexes()
35    return nil
36 }
```

A problem has been detected and your Operation System has been shut down to prevent damage to your computer.



Check to make sure any new hardware or software is properly installed. If this is a new installation, ask your hardware or software manufacturer for and Operation System updates you might need.

Technical information:

*** STOP: 0x000000FE (0x00000008, 0x00000009, 0x847075cc)

Your PC ran into a problem and needs to restart.

We're just collecting some error info, and then we'll restart for you.

VectorStock

VectorStock.com/28701769

Au démarrage, on renaît : on charge le dernier snapshot dans le state, puis on rejoue le journal AOF par-dessus, opération par opération. On reconstruit enfin les index. L'état est strictement identique à avant l'arrêt.

Batch

Plusieurs commandes d'un coup

On exécute plusieurs commandes d'un coup en réutilisant Apply, sans dupliquer la moindre logique.
Ça servira à l'UI web.

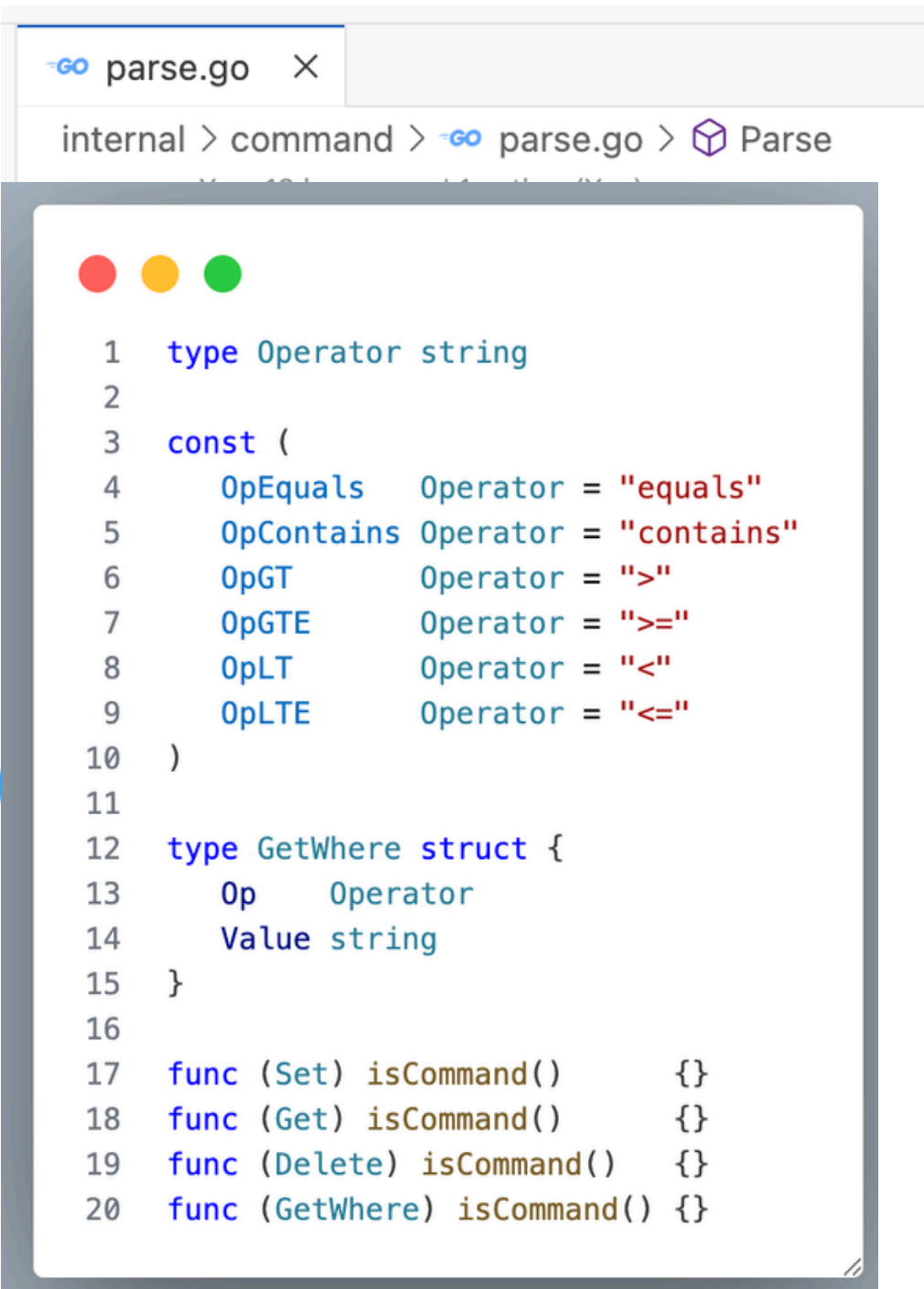
```
1  type BatchResult struct {
2      Result Result
3      Err     error
4  }
5
6  func (e *Engine) Batch(cmds []command.Command) []BatchResult {
7      return lo.Map(cmds, func(cmd command.Command, _ int) BatchResult {
8          res, err := e.Apply(cmd)
9          return BatchResult{Result: res, Err: err}
10     })
11 }
12
```

Le batch exécute plusieurs commandes en un appel, en réutilisant Apply via un lo.Map (permet de transformer une liste en une autre liste). Style fonctionnel, zéro logique dupliquée. Ça devient crucial en WASM : chaque aller-retour JS↔Go coûte cher, donc envoyer 5000 commandes en un message plutôt que 5000 messages change tout.

GET WHERE + index

Interroger par valeur

On ajoute une commande que Redis n'a pas : interroger par valeur, avec des opérateurs comme >, equals ou contains.



```
GO parse.go ×
internal > command > GO parse.go > Parse
1  type Operator string
2
3  const (
4      OpEquals  Operator = "equals"
5      OpContains Operator = "contains"
6      OpGT      Operator = ">"
7      OpGTE     Operator = ">="
8      OpLT      Operator = "<"
9      OpLTE     Operator = "<="
10 )
11
12 type GetWhere struct {
13     Op      Operator
14     Value string
15 }
16
17 func (Set) isCommand() {}
18 func (Get) isCommand() {}
19 func (Delete) isCommand() {}
20 func (GetWhere) isCommand() {}
```

Comprendre la requête

Des index tenus à jour

À chaque écriture on mets à jour deux index. On paie le coût à l'écriture pour lire beaucoup plus vite ensuite.

```
1 func (e *Engine) indexAdd(key, value string) {
2     set := e.equalsIndex[value]
3     if set == nil {
4         set = make(map[string]struct{})
5         e.equalsIndex[value] = set
6     }
7     set[key] = struct{}{}
8
9     if n, ok := parseNumber(value); ok {
10        e.rangeIdx.add(key, n)
11    }
12 }
13
14 func (e *Engine) indexRemove(key, value string) {
15     if set := e.equalsIndex[value]; set != nil {
16         delete(set, key)
17         if len(set) == 0 {
18             delete(e.equalsIndex, value)
19         }
20     }
21
22     if n, ok := parseNumber(value); ok {
23         e.rangeIdx.remove(key, n)
24     }
25 }
26
27 func (e *Engine) rebuildIndexes() {
28     e.equalsIndex = make(map[string]map[string]struct{})
29     e.rangeIdx = newBTree(btreesMinDegree)
30     for key, rec := range e.state {
31         e.indexAdd(key, rec.value)
32     }
33 }
```

La bonne stratégie de recherche

La recherche choisit la bonne stratégie selon l'opérateur : égalité instantanée, plage via l'index, ou scan pour contains.

```
1 func (e *Engine) Filter(q command.GetWhere) []Entry {
2     e.mu.Lock()
3     defer e.mu.Unlock()
4
5     switch q.Op {
6     case command.OpEquals:
7         return e.filterEquals(q.Value)
8     case command.OpContains:
9         return e.filterContains(q.Value)
10    default:
11        return e.filterRange(q.Op, q.Value)
12    }
13 }
14
15 func (e *Engine) filterEquals(value string) []Entry {
16     now := e.clock()
17     var entries []Entry
18     for key := range e.equalsIndex[value] {
19         if rec, ok := e.state[key]; ok && !rec.expired(now) {
20             entries = append(entries, Entry{Key: key, Value: rec.value})
21         }
22     }
23     return entries
24 }
25
26 func (e *Engine) filterContains(sub string) []Entry {
27     now := e.clock()
28     var entries []Entry
29     for key, rec := range e.state {
30         if !rec.expired(now) && strings.Contains(rec.value, sub) {
31             entries = append(entries, Entry{Key: key, Value: rec.value})
32         }
33     }
34     return entries
35 }
36
37 func (e *Engine) filterRange(op command.Operator, target string) []Entry {
38     n, ok := parseNumber(target)
39     if !ok {
40         return nil
41     }
```

Les index permettent à répondre vite aux GET WHERE (chercher par valeur) sans parcourir toute la base : on prépare à l'avance des structures rangées par valeur, pour trouver directement les bonnes clés au lieu de tout scanner.

GO B-Tree

Un B-Tree comme les vraies BDD

On remplace le tableau trié par un B-Tree.

```
1 func (t *bTree) add(key string, n float64) {
2     if entry := t.search(n); entry != nil {
3         entry.keys[key] = struct{}{}
4         return
5     }
6
7     root := t.root
8     if len(root.entries) == 2*t.degree-1 {
9         newRoot := &bTreeNode{children: []*bTreeNode{root}}
10        newRoot.splitChild(0, t.degree)
11        t.root = newRoot
12        newRoot.insertNonFull(n, key, t.degree)
13        return
14    }
15    root.insertNonFull(n, key, t.degree)
16 }
17
18 func (t *bTree) remove(key string, n float64) {
19     if entry := t.search(n); entry != nil {
20         delete(entry.keys, key)
21     }
22 }
23
24 func (t *bTree) query(op command.Operator, target float64) []string {
25     var low, high float64
26     switch op {
27     case command.OpGT, command.OpGTE:
28         low, high = target, math.Inf(1)
29     case command.OpLT, command.OpLTE:
30         low, high = math.Inf(-1), target
31     default:
32         return nil
33     }
34
35     var out []string
36     t.root.rangeScan(low, high, op, target, &out)
37     return out
38 }
39
40 func (t *bTree) search(n float64) *bTreeNode {
41     return t.root.search(n)
42 }
```

Le B-Tree sert à garder les valeurs numériques triées en permanence, pour répondre vite aux requêtes de plage (>, <, >=, <=) sans parcourir toute la base.

TTL & expiration

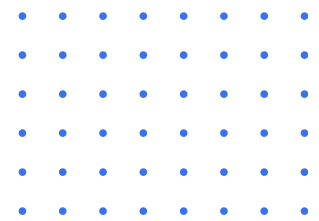
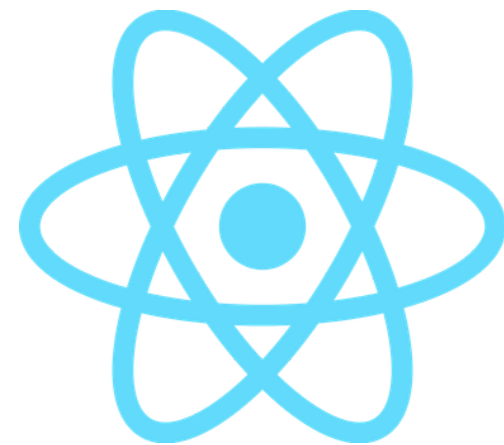
La clé stocke une date d'expiration absolue, pas une durée

Une clé ne stocke plus juste une valeur, mais aussi une date de péremption. Si l'heure actuelle la dépasse, elle est morte.

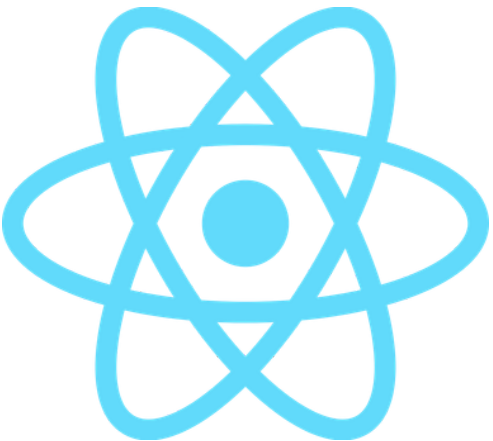
```
1 package engine
2
3 import "time"
4
5 type record struct {
6     value      string
7     expiresAt  time.Time
8 }
9
10 func (r record) expired(now time.Time) bool {
11     return !r.expiresAt.IsZero() && !now.Before(r.expiresAt)
12 }
13
14 func (r record) expiresAtMillis() int64 {
15     if r.expiresAt.IsZero() {
16         return 0
17     }
18     return r.expiresAt.UnixMilli()
19 }
20
21 type persistedRecord struct {
22     Value      string `json:"value"`
23     ExpiresAt  int64  `json:"expires_at,omitempty"`
24 }
25
26 func unixNano(t time.Time) int64 {
27     if t.IsZero() {
28         return 0
29     }
30     return t.UnixNano()
31 }
32
33 func fromUnixNano(n int64) time.Time {
34     if n == 0 {
35         return time.Time{}
36     }
37     return time.Unix(0, n)
38 }
39
```

Le TTL sert à faire expirer une clé automatiquement après un délai (SET x "v" EX 60), pour qu'elle disparaisse toute seule une fois périmée — comme une session ou un cache.

WASM + SDK TypeScript + React



Pourquoi ?



WasmRedis

moteur Go → WASM · persistance OPFS · SDK TypeScript · virtual scroll fait main · render granulaire Legend State (la colonne de droite compte les renders de chaque ligne)

clé

valeur

EX (s)

SET

Seed 1 000

Seed 100 000

🔧 Modifier 1 ligne au hasard

GET

>

▼

50000

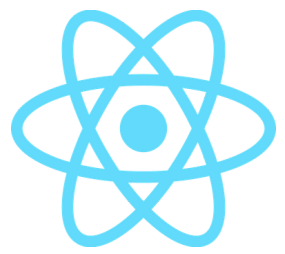
exec

Flush

Snapshot

100 000 entrées – 24 lignes montées dans le DOM (n° 1 à 24)

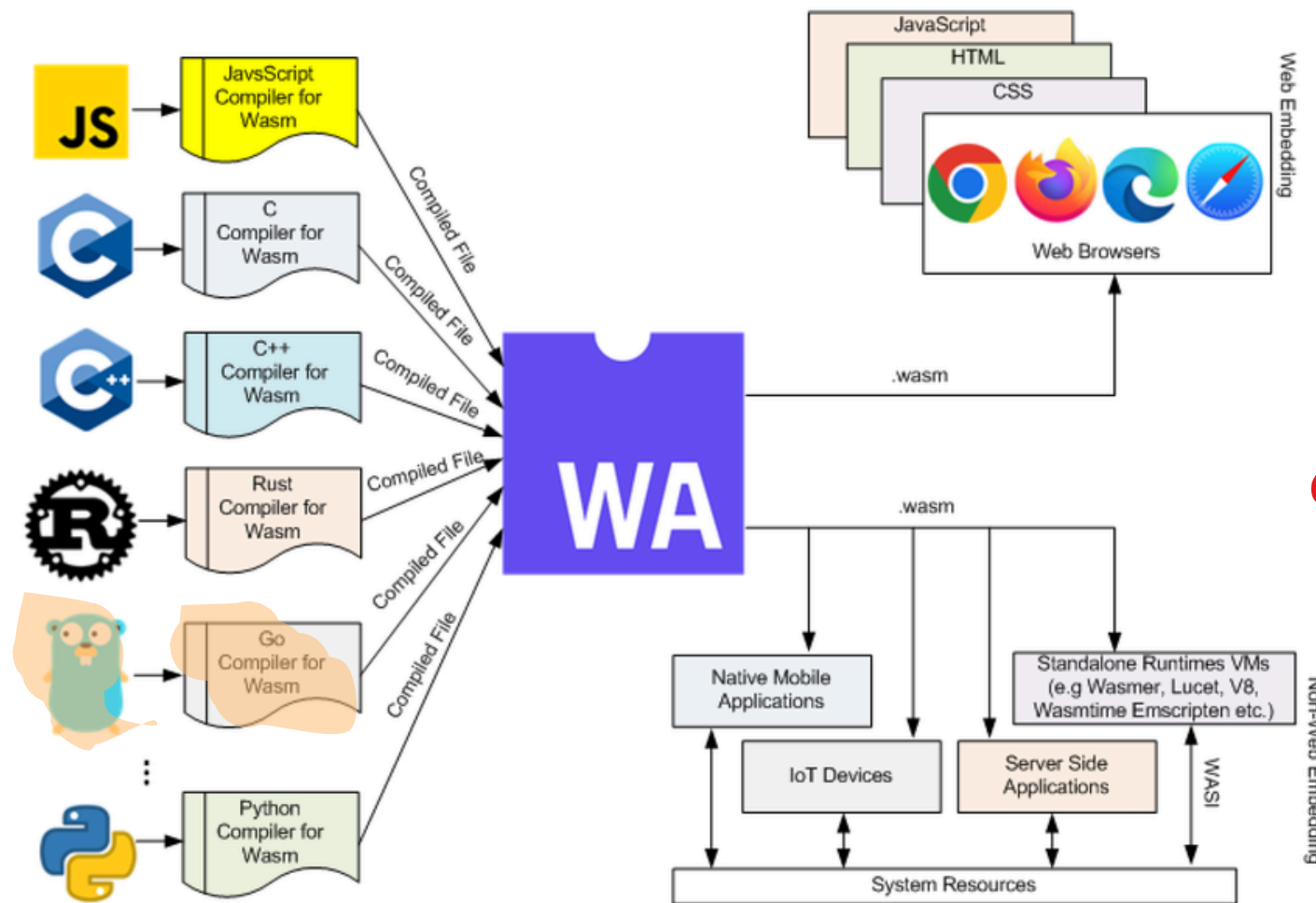
item:00000	60110	1	×
item:00001	44829	1	×
item:00002	85246	1	×
item:00003	66973	1	×
item:00004	17481	1	×
item:00005	52659	1	×
item:00006	27322	1	×
item:00007	emma	1	×
item:00008	86547	1	×
item:00009	47231	1	×
item:00010	24992	1	×
item:00011	88205	1	×
item:00012	74573	1	×
item:00013	30700	1	×
item:00014	19725	1	×
item:00015	emma	1	×



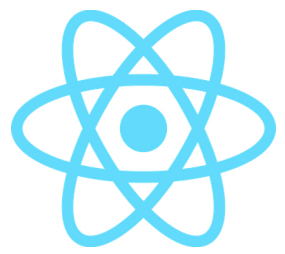
WASM + SDK TypeScript + React

Le moteur Go dans le navigateur

On compile le moteur Go en WebAssembly et on expose ses fonctions au JavaScript : il tourne dans le navigateur, sans serveur.



```
1 func main() {
2   js.Global().Set("__wasmredis", js.ValueOf(map[string]any{
3     "init":    guard(initEngine),
4     "execBatch": guard(execBatch),
5     "getWhere": guard(getWhere),
6     "getMany": guard(getMany),
7     "entries":  guard(entriesHandler),
8     "flush":    guard(flushHandler),
9     "snapshot": guard(snapshotHandler),
10    "close":    guard(closeHandler),
11  }))
12  select {}
13 }
```



WASM + SDK TypeScript + React

Le moteur Go dans le navigateur

On compile le moteur Go en WebAssembly et on expose ses fonctions au JavaScript : il tourne dans le navigateur, sans serveur.



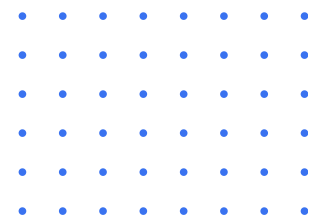
On réutilise la persistance et le TTL des phases précédentes



La persistance navigateur (OPFS) implémente la même interface Storage



Sur un thread séparé, pour ne jamais bloquer l'interface





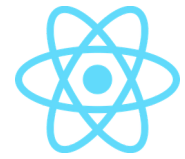
Un SDK TypeScript propre

Le SDK TypeScript transforme chaque appel au worker en une Promise, avec un identifiant pour relier chaque réponse à sa demande.

```
1  return {  
2    request: (msg) =>  
3      new Promise((resolve, reject) => {  
4        const id = nextId++;  
5        pending.set(id, { resolve, reject });  
6        worker.postMessage({ ...msg, id });  
7      }),  
8    terminate: () => worker.terminate(),  
9  };
```

TS client.ts ×

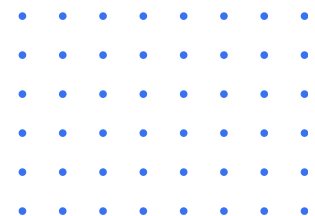
web > src > sdk > TS client.ts > [createWorkerClient](#)

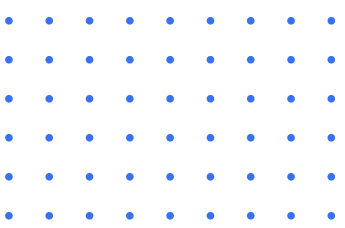


En résumé

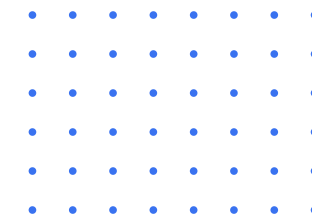
De 500 lignes en Phase 1 à un moteur complet : parsing typé, persistance à deux niveaux, index B-Tree, TTL, et portage WebAssembly.

Un seul cœur métier en Go, réutilisé sur deux plateformes — terminal et navigateur — sans le réécrire.





DEMO LIVE



Fin Présentation

Golang - ReactJS



Merci de m'avoir écouté.

Axel Pointud — M2 ESGI - 16 septembre 2026

Code source : <https://github.com/axel-po/wasm-golang-redis>

